# Modul 6: Generate Data Otomatis & Authentication

## 6.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep authentication, session, hashing password, dan middleware pada Laravel.
2. Menginstal dan menggunakan Laravel Breeze untuk membuat fitur register, login, dan logout secara otomatis.
3. Memahami alur kerja register, login, logout, dan middleware auth di balik layar. 4. Mampu mengimplementasikan database seeding dengan Faker untuk membuat akun pengguna (user) dummy.
5. Mampu membangun fitur authentication (register, login, logout) secara manual tanpa Breeze.
6. Mampu melindungi halaman aplikasi agar hanya dapat diakses oleh pengguna yang sudah login.

## 6.2 Alat dan Bahan

Untuk menjalankan modul ini, diperlukan:

**Perangkat Keras:**

• Komputer atau laptop dengan sistem operasi Windows/Linux/MacOS. • Minimal prosesor 1 GHz, RAM 4GB (disarankan 8GB), serta ruang penyimpanan kosong minimal 2GB.

**Perangkat Lunak:**

- XAMPP atau Laragon sebagai server lokal (Apache & MySQL/MariaDB), termasuk phpMyAdmin untuk mengelola database.
- Composer sebagai dependency manager PHP (sudah terinstal).
- PHP versi 8.2 atau lebih tinggi (sudah termasuk dalam XAMPP versi terbaru). • Laravel Framework yang sudah terinstal melalui Composer (dapat melanjutkan project dari modul sebelumnya).
- Node.js beserta NPM (Node Package Manager) untuk meng-compile asset tampilan Laravel Breeze.
- Teks Editor / IDE seperti Visual Studio Code atau PhpStorm.
- Browser Web seperti Google Chrome atau Mozilla Firefox untuk menjalankan aplikasi.

## 6.3 Tugas Pendahuluan

1. Mengapa aplikasi web membutuhkan authentication?
2. Apa fungsi middleware dalam Laravel?
3. Sebutkan contoh fitur yang biasanya disediakan dalam authentication Laravel! 4. Apa perbedaan antara authentication dan authorization?
5. Apa yang dimaksud dengan session dan bagaimana Laravel mengetahui bahwa seorang pengguna sudah login?
6. Mengapa password tidak boleh disimpan dalam bentuk teks asli di database? Jelaskan apa yang dimaksud dengan hashing!

## 6.4 Materi Praktikum

### 6.4.1 Pengertian Authentication

Authentication (otentikasi) adalah proses untuk memastikan identitas seorang pengguna, yaitu memeriksa apakah pengguna benar-benar orang yang ia akui. Pada aplikasi web, proses ini umumnya dilakukan melalui halaman login dengan memasukkan

email dan password. Agar mudah dibayangkan, authentication ibarat petugas keamanan di pintu masuk kantor yang memeriksa kartu identitas setiap tamu. Hanya orang yang identitasnya valid yang boleh masuk, sedangkan yang tidak memiliki kartu identitas akan ditolak. Authentication sering disamakan dengan authorization (otorisasi), padahal keduanya berbeda:

- **Authentication** → menjawab pertanyaan "Siapa Anda?", misalnya proses login dengan email dan password.
- **Authorization** → menjawab pertanyaan "Apa saja yang boleh Anda lakukan?", misalnya hanya admin yang boleh menghapus data mahasiswa.

Pada modul ini, pembahasan difokuskan pada authentication, yaitu fitur register (mendaftar akun), login (masuk ke aplikasi), dan logout (keluar dari aplikasi).

### 6.4.2 Fungsi Authentication

- Melindungi halaman dan data penting agar tidak dapat diakses oleh sembarang orang. • Mengenali setiap pengguna sehingga aplikasi dapat menampilkan data sesuai pemiliknya, misalnya pada halaman profil.
- Mencatat siapa yang melakukan suatu aksi di dalam aplikasi.
- Menjadi dasar untuk menerapkan authorization (pembagian hak akses) pada tahap pengembangan berikutnya.

### 6.4.3 Konsep Penting dalam Authentication Laravel

Sebelum mulai praktik, pahami terlebih dahulu beberapa istilah yang akan sering muncul pada modul ini:

- **Session** → catatan sementara yang disimpan di server untuk mengingat bahwa seorang pengguna sudah login. Tanpa session, pengguna harus login ulang setiap kali berpindah halaman. **Session** ibarat gelang tiket di taman bermain: cukup diperiksa sekali di pintu masuk, lalu pengunjung bebas berpindah wahana selama gelangnya masih terpasang.
- **Hashing password** → proses mengubah password menjadi deretan karakter acak yang tidak dapat dikembalikan ke bentuk aslinya. Laravel menggunakan **Hash::make()** untuk membuat hash dan **Hash::check()** untuk mencocokkan password saat login, sehingga walaupun database bocor, password asli pengguna tetap tidak diketahui.
- **Middleware** → lapisan penyaring yang dijalankan sebelum permintaan (request) sampai ke controller. **Middleware** auth memeriksa apakah pengguna sudah login; jika belum, pengguna otomatis diarahkan ke halaman login.
- **CSRF token (@csrf)** → token keamanan pada setiap form agar data hanya dapat dikirim dari aplikasi kita sendiri, bukan dari situs lain.
- **Facade Auth** → fitur bawaan Laravel untuk mengelola authentication, misalnya **Auth::attempt()** (mencoba login), **Auth::login()** (login secara langsung), **Auth::logout()** (keluar), dan **Auth::user()** (mengambil data pengguna yang sedang login).

### 6.4.4 Mengenal Laravel Breeze

Laravel Breeze adalah starter kit (paket awal) resmi dari Laravel yang menyediakan fitur authentication lengkap secara otomatis, meliputi register, login, logout, lupa password, verifikasi email, dan edit profil. Dengan Breeze, kita tidak perlu menulis route, controller, dan view authentication dari awal, cukup menjalankan beberapa perintah di terminal.

Tampilan Breeze
dibuat menggunakan Tailwind CSS dan di-compile menggunakan Vite, sehingga dibutuhkan Node.js dan NPM. Pada modul ini, authentication dipelajari dengan dua cara, yaitu menggunakan Breeze dan secara manual tanpa Breeze. Dengan mempelajari keduanya, Anda tidak hanya dapat membuat fitur login dengan cepat, tetapi juga memahami proses yang terjadi di balik layar.

### 6.4.5 Instalasi Laravel Breeze

Pada subbab ini, kita akan memasang Laravel Breeze pada project Laravel. Ikuti langkah langkah di bawah ini secara berurutan dan pastikan setiap perintah berhasil dijalankan sebelum melanjutkan ke langkah berikutnya.

1. Buka terminal di folder project Laravel Anda, lalu instal package Laravel Breeze dengan perintah berikut:

```
PS C:\xampp\htdocs\laravel> composer require larave
```

Perintah composer require mengunduh package Laravel Breeze ke dalam folder vendor.

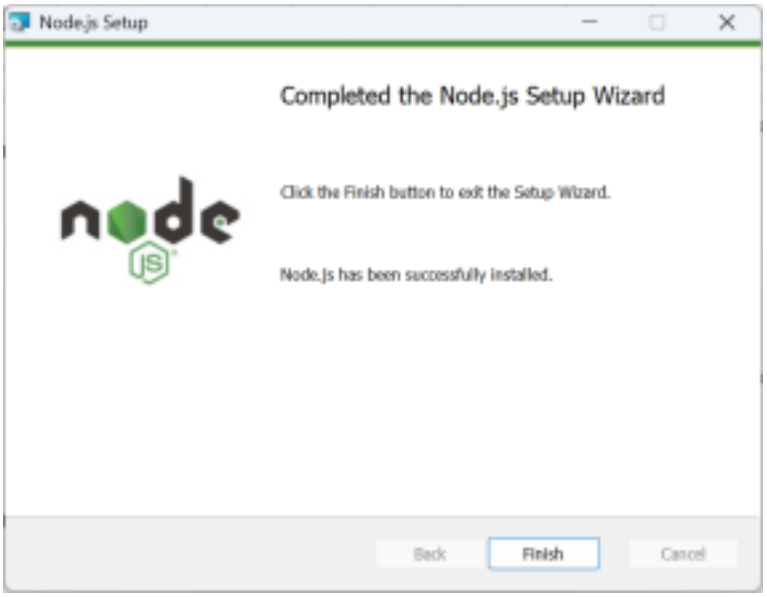
2. Pasang Breeze dengan tampilan Blade menggunakan perintah berikut:

```
PS C:\xampp\htdocs\laravel> php artisan breeze:install blade
```

Perintah breeze:install blade membuat route, controller, view, dan file pendukung authentication secara otomatis. Pilihan blade berarti tampilan dibuat menggunakan Blade template (bukan React atau Vue). Jika muncul pertanyaan tambahan di terminal, misalnya pilihan dark mode atau framework testing, pilih jawaban bawaan dengan menekan Enter.

Perhatian: perintah ini akan menimpa (overwrite) file **routes/web.php** serta beberapa file view, seperti **resources/views/layouts/app.blade.php**. Jika project Anda sudah berisi route dan view dari modul sebelumnya, salin (backup) terlebih dahulu file-file tersebut atau gunakan project Laravel baru agar pekerjaan sebelumnya tidak hilang.

3. Jika Node.js belum terinstal, unduh di **https://nodejs.org/en/download/**, jalankan installer-nya, lalu klik Finish (Sudah dilakukan di modul 1).



Node.js adalah lingkungan untuk menjalankan JavaScript di luar browser, sedangkan NPM (Node Package Manager) otomatis ikut terinstal bersama Node.js. Fungsi NPM mirip dengan Composer, bedanya NPM mengelola package JavaScript dan CSS. Untuk memastikan instalasi berhasil, tutup lalu buka kembali terminal, kemudian jalankan
perintah node -v dan npm -v. Jika muncul nomor versi, berarti Node.js siap digunakan. Jika muncul galat, lihat subbab 6.4.9.

4. Setelah Node.js terinstal, jalankan perintah berikut untuk mengunduh package front-end yang dibutuhkan Breeze:

```
PS C:\xampp\htdocs\laravel> npm install
```

Setelah proses **npm install** selesai, jalankan perintah berikut untuk meng-compile asset (CSS dan JavaScript):

```
PS C:\xampp\htdocs\laravel> npm run dev
```

- **npm install** → mengunduh seluruh package JavaScript yang dibutuhkan Breeze ke dalam folder node_modules. Perintah ini cukup dijalankan satu kali.
- **npm run dev** → menjalankan Vite untuk meng-compile file CSS (Tailwind) dan JavaScript, sekaligus memantau perubahan file secara otomatis.

Perintah **npm run dev** harus tetap berjalan selama proses pengembangan agar tampilan halaman login dan register dapat dimuat dengan benar. Oleh karena itu, jangan tutup terminal ini.

5. Karena terminal pertama sedang digunakan oleh **npm run dev**, buka terminal baru atau buat split terminal di Visual Studio Code dengan mengklik tombol berikut:



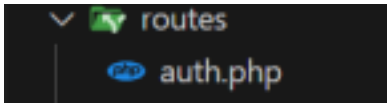

Dengan split terminal, Anda dapat menjalankan dua perintah sekaligus, yaitu terminal pertama untuk **npm run dev** dan terminal kedua untuk perintah php artisan.

6. Pada terminal yang baru (terminal kedua), jalankan migration dengan perintah berikut:

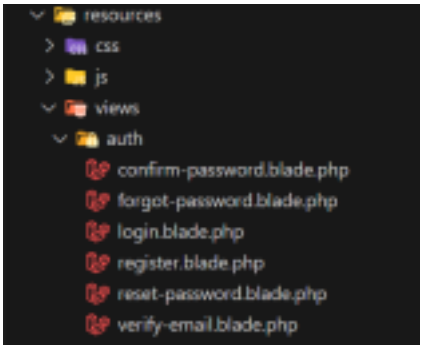
```
PS C:\xampp\htdocs\laravel> php artisan migrate
```

Perintah ini membuat tabel yang dibutuhkan untuk authentication, di antaranya users (menyimpan data akun), password_reset_tokens (menyimpan token reset password), dan sessions (menyimpan data sesi login). Jika tabel tersebut sudah ada dari modul sebelumnya, terminal akan menampilkan pesan Nothing to migrate, dan hal tersebut wajar. Pastikan konfigurasi database pada file **.env** sudah benar seperti pada Modul 4.

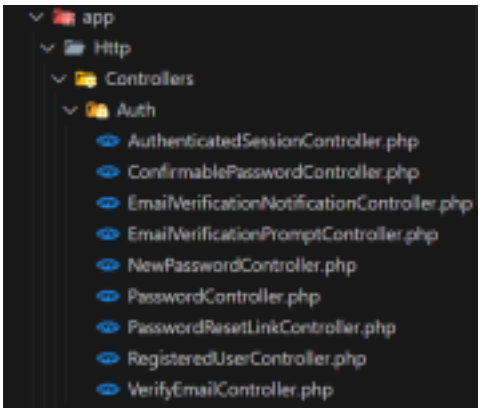
7. Perhatikan beberapa file yang dibuat secara otomatis oleh Breeze berikut: • **Route authentication pada file routes/auth.php** → berisi alamat **/register**, **/login**, **/logout**, dan route authentication lainnya.




- **View Blade pada folder resources/views/auth** → berisi tampilan halaman login, register, lupa password, dan lainnya.




- **Controller pada folder app/Http/Controllers/Auth** → berisi logika proses register, login, dan logout.




Ketiga bagian tersebut membentuk pola MVC seperti pada modul sebelumnya: route menentukan alamat URL, controller memproses data, dan view menampilkan halaman. Bedanya, seluruh file tersebut sudah dibuat oleh Breeze sehingga kita cukup mempelajari dan menggunakannya.

### 6.4.6 Routing dan Pengujian Authentication

Setelah instalasi selesai, Breeze secara otomatis mengubah isi file **routes/web.php** menjadi seperti berikut:

```
<?php

use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
 return view('welcome');
});

Route::get('/dashboard', function () {
 return view('dashboard');
})->middleware(['auth',

'verified'])->name('dashboard'); require

__DIR__.'/auth.php';
```

- **require __DIR__.'/auth.php';** → memuat seluruh route authentication dari file **routes/auth.php** (login, register, logout, dan lainnya), sehingga file **web.php** tetap ringkas.
- Route::get('/dashboard', ...) → menampilkan view dashboard.blade.php ketika alamat /dashboard dibuka.
- **->middleware(['auth', 'verified'])** → halaman dashboard hanya dapat diakses oleh pengguna yang sudah login (auth) dan sudah memverifikasi email (verified). Secara bawaan, model User belum mewajibkan verifikasi email, sehingga pengguna yang baru mendaftar tetap dapat membuka dashboard.
- **->name('dashboard')** → memberi nama pada route sehingga dapat dipanggil dengan route('dashboard').
- use App\Http\Controllers\ProfileController; → mengimpor controller profil bawaan Breeze yang digunakan untuk fitur edit profil.

Selanjutnya, uji fitur authentication yang telah dibuat dengan langkah-langkah berikut:

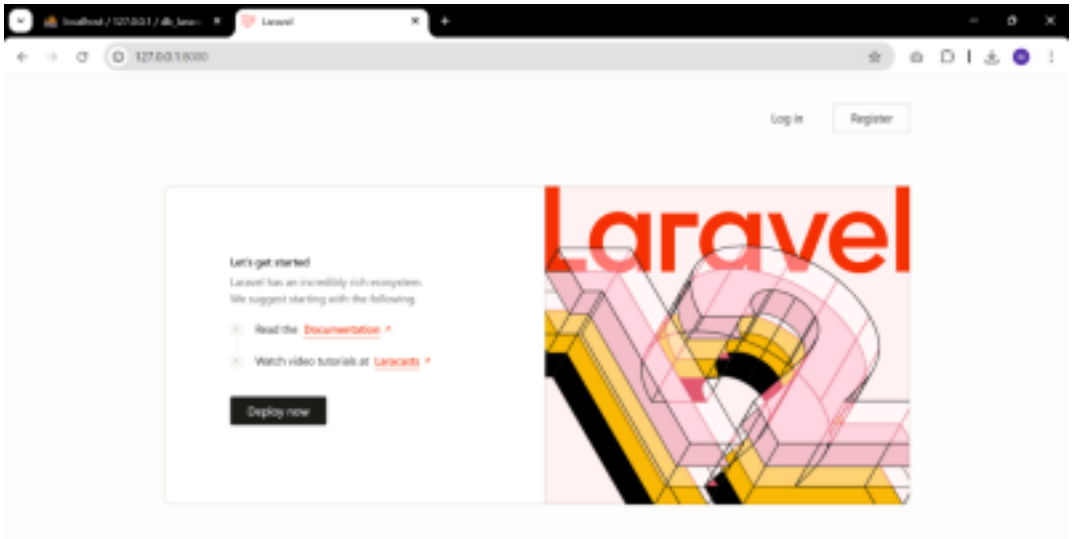
1. Pada terminal kedua, jalankan server Laravel dengan perintah **php artisan serve**, lalu klik atau buka alamat **http://127.0.0.1:8000** di browser.

```
PS C:\xampp\htdocs\laravel> php artisan serve
```

Pastikan terminal pertama (**npm run dev**) masih berjalan. Jika tampilan halaman berantakan atau muncul galat Vite manifest not found, berarti perintah **npm run dev** belum dijalankan atau sudah terhenti atau bisa juga dilakuakn dengan sintax **composer run dev** (jika menggunakan composer run dev, tidak perlu menjalankan **php artisan serve** & **npm run dev**).

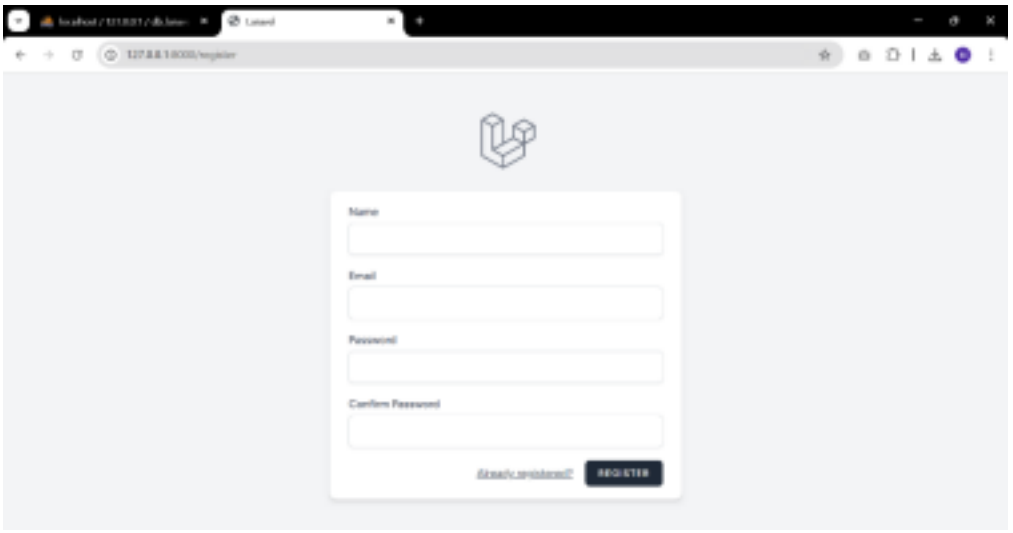
```
asad@asad UCRT64 /e/Tel-u/SIKC/2026
$ composer run dev
```

2. Pada halaman awal (welcome), terdapat tombol Log in dan Register di pojok kanan



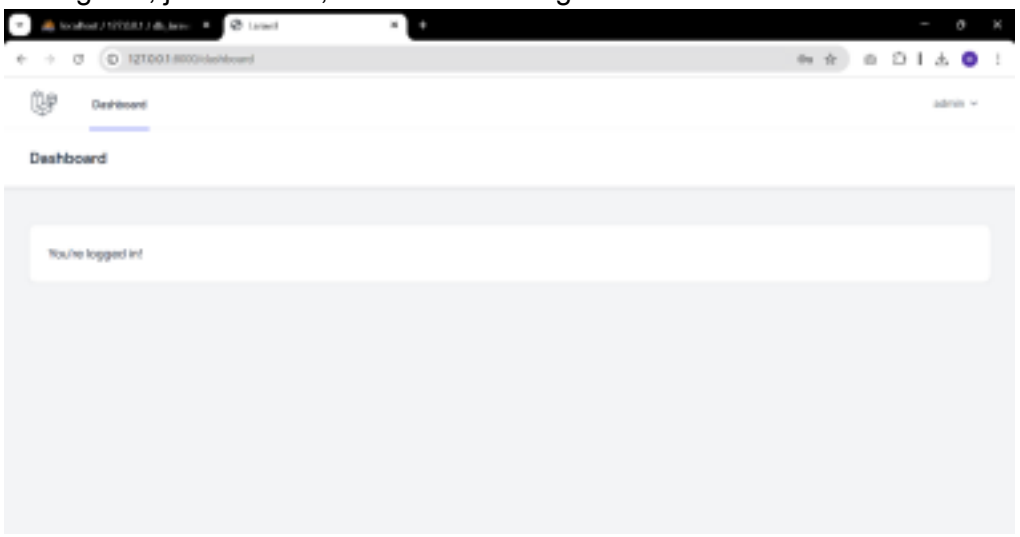

atas.

3. Klik Register, lalu lengkapi form dengan nama, email, password, dan konfirmasi password.



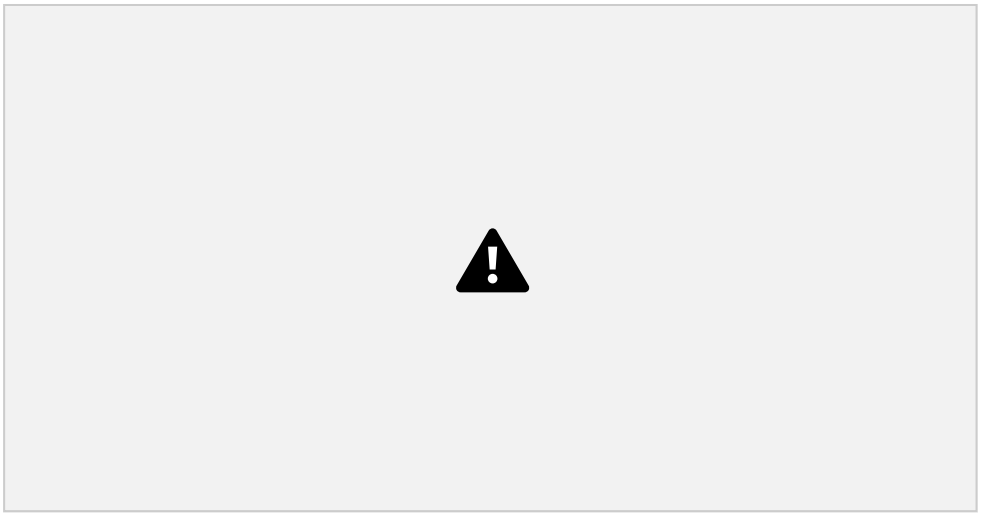
Password minimal terdiri dari 8 karakter, dan isi kolom Confirm Password harus sama dengan Password. Jika ada data yang tidak sesuai, Breeze akan menampilkan pesan galat di bawah kolom tersebut. Pesan ini merupakan hasil validasi yang dilakukan oleh controller.

4. Klik Register; jika berhasil, Anda otomatis login dan diarahkan ke dashboard.



Pada halaman dashboard terdapat menu nama pengguna di pojok kanan atas yang berisi pilihan Profile dan Log Out. Cobalah klik Log Out, lalu login kembali melalui menu Log in menggunakan email dan password yang baru saja didaftarkan.

5. Buka phpMyAdmin dan periksa tabel users. Akun yang baru didaftarkan akan muncul di sana.



Perhatikan kolom password: isinya berupa deretan karakter acak yang diawali **$2y**$, bukan password asli yang Anda ketik. Inilah hasil hashing menggunakan algoritma bcrypt, sehingga password tetap aman walaupun isi database dilihat oleh orang lain.

### 6.4.7 Alur Kerja Authentication pada Breeze

Agar lebih memahami apa yang terjadi di balik layar, berikut alur kerja setiap fitur authentication yang dibuat oleh Breeze, mulai dari pengguna membuka halaman hingga diarahkan ke halaman tujuan:

**a. Register (Pendaftaran Akun)**

- **User buka halaman** → GET **/register**.
- Breeze load view dari **resources/views/auth/register.blade.php**.
- **User isi form** → kirim ke POST **/register**.
- Controller **RegisteredUserController** menerima request, memvalidasi input (nama, email, password).
- **Jika valid** → buat user baru di tabel users dengan password yang di-*hash*. • Laravel langsung login user tersebut.
- User diarahkan ke **/dashboard**.

**b. Login**

- **User buka halaman** → GET **/login**.
- View **resources/views/auth/login.blade.php** ditampilkan.
- **User isi form (email & password)** → kirim ke POST **/login**.
- Controller **AuthenticatedSessionController** akan:
  - ➢ Cek apakah email ada di tabel users.
  - ➢ Verifikasi password dengan **Hash::check()**.
- **Jika berhasil** → Laravel set session auth untuk user tersebut.
- User diarahkan ke **/dashboard**.

**c. Middleware auth**

- Setelah login, user coba akses **/dashboard**.
- Route **/dashboard** dilindungi dengan **->middleware(['auth', 'verified'])**. • Middleware auth akan cek:
  - ➢ Apakah user sudah login?
  - ➢ **Kalau iya** → lanjut ke view **dashboard.blade.php**.
  - ➢ **Kalau belum** → redirect ke **/login**.

**d. Logout**

- **Pada dashboard ada tombol Logout** → mengarah ke POST **/logout**.
- Controller **AuthenticatedSessionController@**destroy dipanggil.
- Laravel hapus session login.
- User diarahkan ke halaman utama.

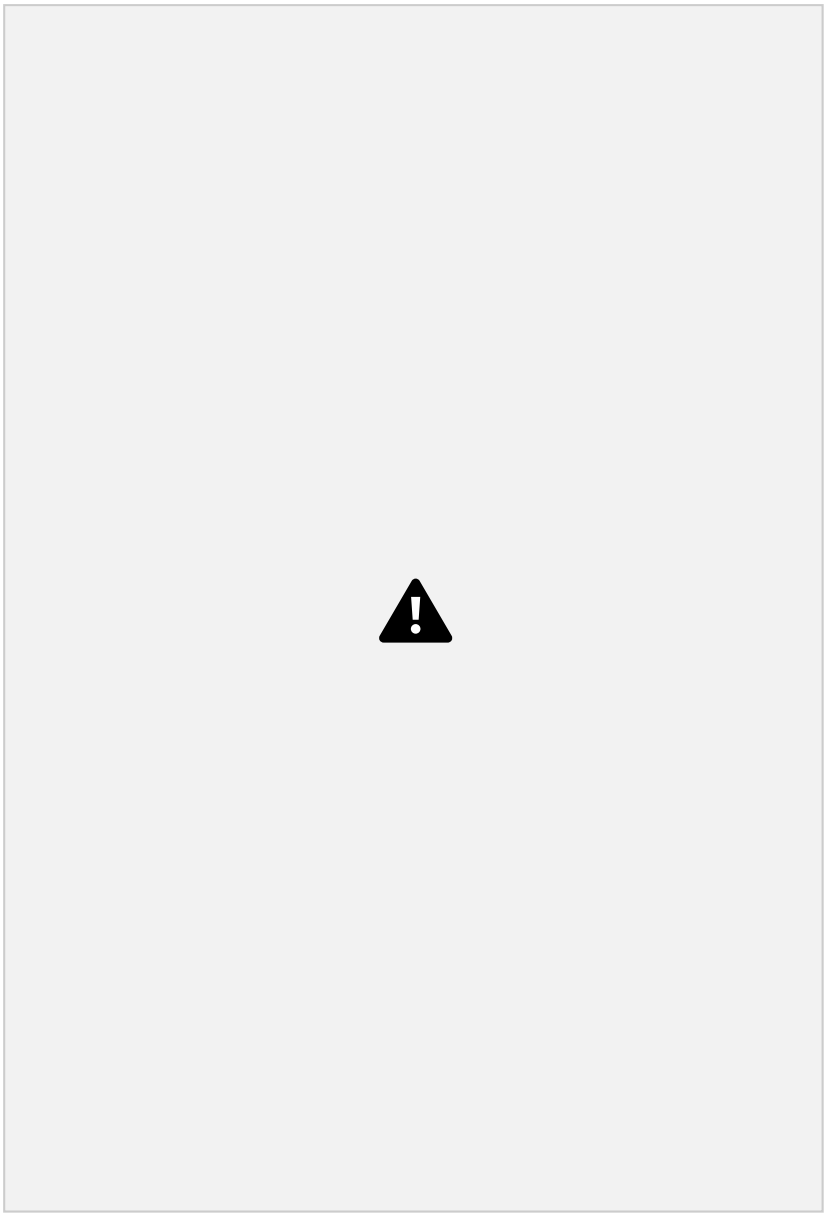
Secara singkat, register membuat akun baru sekaligus membuat session login, login memeriksa email dan password lalu membuat session, middleware auth memeriksa session sebelum halaman dibuka, dan logout menghapus session tersebut.

### 6.4.8 Membuat User Dummy dengan Seeder dan Faker

Pada Modul 4, Anda telah menggunakan seeder dan Faker untuk mengisi tabel mahasiswa secara otomatis. Teknik yang sama dapat digunakan untuk membuat akun pengguna (user) dummy, sehingga kita tidak perlu mendaftar satu per satu melalui halaman Register setiap kali ingin menguji fitur login. Perbedaannya, kolom password pada tabel users wajib disimpan dalam bentuk hash. Jika password disimpan sebagai teks biasa, proses login akan selalu gagal karena Laravel mencocokkan password yang diketik dengan nilai hash yang ada di database.

1. Buat seeder baru bernama **UserSeeder** dengan menjalankan perintah berikut di terminal:

   Perintah ini membuat file baru **database/seeders/UserSeeder.php** yang berisi kerangka class seeder dengan method run().
2. Buka file **database/seeders/UserSeeder.php**, lalu ganti isinya dengan kode

berikut: 

- $faker->unique()->safeEmail → menghasilkan email acak yang dijamin tidak kembar, sehingga tidak terjadi galat duplicate entry (kolom email pada tabel users bersifat unique).
- **Hash::make('password')** → seluruh user dummy menggunakan password "password", tetapi tetap disimpan dalam bentuk hash agar aman dan dapat digunakan untuk login.

• use Faker\Factory as Faker; → mengimpor library Faker dengan nama alias Faker. • **Faker::create()** → membuat objek Faker. Karena tanpa locale, nama yang dihasilkan berbahasa Inggris. Gunakan **Faker::create('id_ID')** jika ingin menghasilkan nama Indonesia.

- **for ($i = 0; $i < 5; $i++)** → mengulang proses sebanyak 5 kali sehingga terbentuk 5 akun dummy.
- **DB::table('users')->insert([...])** → memasukkan satu baris data ke tabel users menggunakan Query Builder.
- **Akun admin manual** → akun tetap dengan email **admin@example.com** dan

password **admin123** yang mudah diingat untuk keperluan pengujian.

3. Jalankan seeder dengan perintah berikut:

```
```

Jika berhasil, terminal menampilkan pesan INFO Seeding database dan tabel users akan bertambah 6 data (5 akun dummy dan 1 akun admin). Jika muncul galat Duplicate entry '**admin@example.com**', berarti seeder sudah pernah dijalankan sebelumnya. Hapus data admin tersebut melalui phpMyAdmin, lalu jalankan kembali seeder.

4. Jalankan kembali server dengan perintah **php artisan serve**, lalu uji login menggunakan salah satu email akun dummy (lihat di tabel users pada phpMyAdmin) dengan password password, atau menggunakan akun admin dengan email **admin@example.com** dan password **admin123**. Jika login berhasil dan Anda diarahkan ke halaman dashboard, berarti akun hasil seeder dapat digunakan sama seperti akun yang didaftarkan melalui halaman Register.

### 6.4.9 Mengatasi Kendala Instalasi Node.js dan NPM

Berikut beberapa kendala yang sering muncul saat menginstal atau menjalankan Node.js dan NPM beserta cara mengatasinya:

- Perintah node atau npm tidak dikenali (muncul pesan is not recognized). Pastikan folder instalasi Node.js (biasanya **C:\Program Files\nodejs**) sudah ditambahkan ke Path pada Environment Variables Windows. Caranya, buka menu Edit the system environment variables, klik Environment Variables, pilih Path, klik Edit, lalu tambahkan folder tersebut. Setelah itu, tutup dan buka kembali terminal.
- Muncul galat npm : File **C:\Program Files\nodejs**\npm.ps1 cannot be loaded because running scripts is disabled on this system. Galat ini terjadi karena PowerShell memblokir eksekusi script. Jalankan perintah berikut di terminal, lalu ulangi perintah **npm install**:

```
```

- Perintah **Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass** hanya mengizinkan eksekusi script pada jendela terminal yang sedang dibuka. Jika terminal ditutup, perintah ini perlu dijalankan kembali sebelum menggunakan npm.

### 6.4.10 Membuat Authentication Tanpa Breeze (Manual)

Breeze memudahkan pembuatan authentication, tetapi seluruh kodenya dibuat secara otomatis sehingga proses di baliknya kurang terlihat. Pada subbab ini, kita akan membuat fitur register, login, dan logout secara manual agar Anda memahami cara kerja authentication Laravel dari awal. Beberapa fitur yang digunakan antara lain facade Auth untuk proses login dan logout, Hash untuk mengacak password, validasi pada Request, serta middleware auth untuk melindungi halaman.

**Penting:** kerjakan subbab ini pada project yang belum diinstal Breeze, misalnya project Laravel baru. Jika dikerjakan pada project yang sama dengan Breeze, route dan view akan saling bertabrakan, misalnya karena keduanya sama-sama memiliki route bernama login dan register. Secara garis besar, aplikasi yang akan dibuat bekerja dengan alur sebagai berikut:

• Pengguna membuka **/register**, mengisi form, lalu **RegisterController** menyimpan akun baru ke tabel users dan langsung melakukan login.
• Pengguna membuka **/login**, mengisi email dan password, lalu **LoginController** mencocokkan data tersebut dengan tabel users menggunakan **Auth::attempt()**. • Jika login berhasil, pengguna dapat membuka halaman yang dilindungi middleware auth, seperti **/dashboard** dan **/profile**.
• Ketika tombol Logout diklik, **LoginController** menghapus session sehingga pengguna harus login kembali untuk membuka halaman tersebut.

Ikuti langkah-langkah di bawah ini secara berurutan dan pastikan setiap file disimpan sebelum melanjutkan ke langkah berikutnya.

1. Pastikan model dan migration User sudah tersedia di project Anda. Secara bawaan, Laravel sudah menyediakan file **app/Models/User.php** dan migration untuk tabel users. Pastikan tabel users sudah terbentuk di database dengan menjalankan perintah **php artisan migrate**. Tabel inilah yang akan digunakan untuk menyimpan data akun pengguna.
2. Buat folder baru bernama Auth di dalam folder **app/Http/Controllers**. Folder ini akan berisi dua controller, yaitu **RegisterController** untuk proses pendaftaran dan **LoginController** untuk proses login dan logout.
3. Buat file **app/Http/Controllers/Auth/RegisterController.php**, lalu isi dengan kode di bawah ini dan simpan (Ctrl + S).

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Auth\Events\Registered; // Digunakan untuk memicu event
setelah registrasi berhasil
use Illuminate\Http\Request; // Untuk menangani data yang dikirim dari form
(seperti nama, email, password)
use Illuminate\Support\Facades\Auth; // Untuk proses login secara otomatis use
Illuminate\Support\Facades\Hash; // Untuk mengenkripsi (mengacak) password use
Illuminate\Validation\Rules; // Untuk aturan validasi password

class RegisterController extends Controller
{
  public function create()
  {
    return view('auth.register'); // Menampilkan file view
  auth/register.blade.php }

  public function store(Request $request)
  {
    $request->validate([
      'name' => ['required', 'string', 'max:255'],
      'email' => ['required', 'string', 'email', 'max:255', 'unique:users'],
      'password' => ['required', 'confirmed', Rules\Password::defaults()],
```

```
        ]);

        // Membuat user baru di database dengan data yang sudah divalidasi
        $user = User::create([
          'name' => $request->name, // Ambil nama dari form
          'email' => $request->email, // Ambil email dari form
          'password' => Hash::make($request->password), // Hash password agar
    tidak tersimpan dalam bentuk asli (demi keamanan)
        ]);

        // Memicu event bahwa user baru telah berhasil mendaftar
        event(new Registered($user));

        // Langsung login user tersebut setelah mendaftar
        Auth::login($user);

        // Redirect ke halaman dashboard
        return redirect()->route('dashboard');
      }
    }
```

- **create()** → menampilkan halaman form register (**resources/views/auth/register.blade.php**).
- **store(Request $request)** → menerima data dari form register, memvalidasinya, lalu menyimpan akun baru.
- $request->validate([...]) → memeriksa input: nama wajib diisi, email harus berformat email dan belum terdaftar (unique:users), serta password wajib dikonfirmasi (confirmed) dan memenuhi aturan bawaan (minimal 8 karakter).
- **User::create([...])** → menyimpan data akun baru ke tabel users melalui model User. • **Hash::make($request->password)** → mengacak password sebelum disimpan sehingga password asli tidak tersimpan di database.
- **event(new Registered($user))** → memberi tahu Laravel bahwa ada pengguna baru yang mendaftar, misalnya untuk keperluan pengiriman email verifikasi.
- **Auth::login($user)** → langsung melakukan login untuk pengguna yang baru mendaftar, lalu redirect()->route('dashboard') mengarahkannya ke halaman dashboard.

Aturan confirmed pada password berarti form wajib memiliki input bernama **password_confirmation** yang isinya sama dengan password. Jika validasi gagal, Laravel otomatis kembali ke halaman form sambil membawa pesan galat.

4. Buat file **app/Http/Controllers/Auth/LoginController.php**, lalu isi dengan kode di bawah ini dan simpan.

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request; // Untuk menangani data dari form login use
Illuminate\Support\Facades\Auth; // Fitur Laravel untuk proses login, logout, dan
```

autentikasi

```
class LoginController extends Controller
{
  public function create()
  {
    return view('auth.login'); // Menampilkan file view
  auth/login.blade.php }

  public function store(Request $request)
  {
    // Validasi input pengguna: email dan password harus diisi dan sesuai
    format $credentials = $request->validate([
      'email' => ['required', 'email'],
      'password' => ['required'],
    ]);

    // Jika berhasil, dan checkbox "ingat saya" dicentang, maka sesi akan tetap
    aktif if (Auth::attempt($credentials, $request->remember)) {
      $request->session()->regenerate();

      return redirect()->intended('dashboard'); // Redirect ke
    Dashboard }

    // Jika gagal login
    return back()->withErrors([
      'email' => 'Email atau password yang Anda masukkan salah.',
    ]);
  }

  public function destroy(Request $request)
  {
    Auth::logout(); // Hapus status login

    $request->session()->invalidate(); // Kosongkan semua data session
    $request->session()->regenerateToken(); // Ganti token CSRF agar lebih
    aman

    return redirect('/'); // Redirect ke welcome
  }
}
```

- $credentials = $request->validate([...]) → memastikan email dan password diisi, lalu menyimpannya ke variabel $credentials.
- **Auth::attempt($credentials, $request->remember)** → mencari pengguna berdasarkan email, lalu mencocokkan password menggunakan hash. Hasilnya true jika cocok dan false jika tidak. Parameter kedua digunakan untuk fitur Remember Me.
- $request->session()->regenerate() → membuat ID session baru setelah login untuk mencegah serangan session fixation.
- **redirect()->intended('dashboard')** → mengarahkan pengguna ke halaman

yang sebelumnya ingin dibuka, atau ke dashboard jika tidak ada.

- **back()->withErrors([...])** → kembali ke halaman login sambil menampilkan pesan galat jika email atau password salah.
- **destroy()** → proses logout: **Auth::logout()** menghapus status login, invalidate() mengosongkan data session, dan regenerateToken() mengganti token CSRF. 5. Buat **HomeController** yang akan menampilkan halaman dashboard dan profil dengan menjalankan perintah berikut di terminal: **php artisan make:controller HomeController**. Perintah ini membuat file **app/Http/Controllers/HomeController.php** beserta kerangka class-nya. 6. Buka file **app/Http/Controllers/HomeController.php**, lalu ganti seluruh isinya dengan kode di bawah ini dan simpan.

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class HomeController extends
Controller {
   public function dashboard()
   {
      return view('dashboard');
   }

   public function profile()
   {
      return view('profile');
   }
}
```

- **dashboard()** → menampilkan view **dashboard.blade.php**.
- **profile()** → menampilkan view **profile.blade.php** yang berisi data pengguna yang sedang login.

7. Buka file **routes/web.php**, lalu ubah isinya menjadi seperti di bawah ini dan simpan. Route dashboard dan profile dimasukkan ke dalam grup middleware auth agar hanya dapat diakses setelah pengguna login.

```
<?php
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Auth\RegisterController;
use App\Http\Controllers\Auth\LoginController;
use App\Http\Controllers\HomeController;

Route::get('/', function () {
   return view('welcome'); // Route langsung Views tanpa controller
});

// Auth Routes
```

```
Route::get('/register', [RegisterController::class,
'create'])->name('register'); Route::post('/register',
[RegisterController::class, 'store']);

Route::get('/login', [LoginController::class, 'create'])->name('login');
Route::post('/login', [LoginController::class, 'store']);
Route::post('/logout', [LoginController::class, 'destroy'])->name('logout');

// Protected Routes
Route::middleware('auth')->group(function () {
  Route::get('/dashboard', [HomeController::class, 'dashboard'])-
>name('dashboard');
  Route::get('/profile', [HomeController::class,
'profile'])->name('profile'); });
```

- Route::get('/register', ...) dan Route::get('/login', ...) → menampilkan halaman form register dan login.
- Route::post('/register', ...) dan Route::post('/login', ...) → menerima data yang dikirim dari form menggunakan method POST.
- Route::post('/logout', ...) → menjalankan proses logout. Logout menggunakan POST (bukan GET) agar tidak dapat dipicu hanya dengan membuka sebuah link. • **->name('login')** → nama route login wajib ada, karena middleware auth akan mengarahkan pengguna yang belum login ke route bernama login.
- Route::middleware('auth')->group(function () {...}) → seluruh route di dalam grup ini hanya dapat diakses oleh pengguna yang sudah login.

8. Buka file **resources/views/layouts/app.blade.php** (buat file tersebut jika belum ada), lalu ganti seluruh isinya dengan kode di bawah ini untuk menambahkan menu navigasi.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  initial-scale=1.0"> <meta http-equiv="X-UA-Compatible"
  content="ie=edge">
  <title>@yield('title', 'Auth Project')</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font
awesome/6.4.0/css/all.min.css">
  <meta name="csrf-token" content="{{ csrf_token() }}">
</head>
<body>
  <!-- Navbar (menu bagian atas website) -->
  <nav class="navbar navbar-expand-lg navbar-light bg-light mb-4">
    <div class="container">
      <a class="navbar-brand" href="/">Auth Project</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
```

```
data bs-target="#navbarNav">
          <span class="navbar-toggler-icon"></span>
        </button>

        <div class="collapse navbar-collapse" id="navbarNav">
          <ul class="navbar-nav me-auto">
          </ul>
          <!-- Bagian kanan navbar -->
          <ul class="navbar-nav ms-auto">
            @auth
              <!-- Jika pengguna sudah login, tampilkan menu berikut -->
              <li class="nav-item">
                <a class="nav-link" href="{{ route('dashboard') }}">
                  <i class="fas fa-tachometer-alt me-1"></i>Dashboard
                </a>
              </li>
              <li class="nav-item">
                <a class="nav-link" href="{{ route('profile') }}">
                  <i class="fas fa-user me-1"></i>Profile
                </a>
              </li>
              <li class="nav-item">
                <!-- Tombol logout dengan form (POST) untuk keluar dari
                akun --> <form method="POST" action="{{ route('logout') }}">
                  @csrf
                  <button type="submit" class="nav-link btn btn-link">
                    <i class="fas fa-sign-out-alt me-1"></i>Logout
                  </button>
                </form>
              </li>
            @else
              <!-- Jika pengguna belum login, tampilkan menu Login dan
              Register --> <li class="nav-item">
                <a class="nav-link" href="{{ route('login') }}">
                  <i class="fas fa-sign-in-alt me-1"></i>Login
                </a>
              </li>
              <li class="nav-item">
                <a class="nav-link" href="{{ route('register') }}">
                  <i class="fas fa-user-plus me-1"></i>Register
                </a>
              </li>
            @endauth
          </ul>
        </div>
      </div>
    </nav>

    <div class="container">
      @if(session('success'))
        <div class="alert alert-success alert-dismissible fade show mt-3"
```

```
            role="alert"> {{ session('success') }}
            <button type="button" class="btn-close" data-bs-dismiss="alert"
aria label="Close"></button>
          </div>
        @endif

        @if(session('error'))
          <div class="alert alert-danger alert-dismissible fade show mt-3"
            role="alert"> {{ session('error') }}
            <button type="button" class="btn-close" data-bs-dismiss="alert"
aria label="Close"></button>
          </div>
        @endif

        @if($errors->any())
          <div class="alert alert-danger alert-dismissible fade show mt-3"
            role="alert"> <ul class="mb-0">
              @foreach($errors->all() as $error)
                <li>{{ $error }}</li>
              @endforeach
            </ul>
            <button type="button" class="btn-close" data-bs-dismiss="alert"
aria label="Close"></button>
          </div>
        @endif

        @yield('content')
      </div>

      <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
      <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
< /script>
      @stack('scripts')
</body>
</html>
```

- **@yield('content')** → tempat isi halaman dari view lain (register, login, dashboard, profile) ditampilkan.
- **@auth ... @else ... @endauth** → menampilkan menu Dashboard, Profile, dan Logout jika pengguna sudah login, serta menu Login dan Register jika belum login. • <form method="POST" action="{{ route('logout') }}"> → tombol Logout dibuat dalam bentuk form karena route logout menggunakan method POST.
- **@if(session('success')) dan @if($errors->any())** → menampilkan pesan berhasil atau pesan galat validasi di bagian atas halaman.
- **Link CSS dan JavaScript Bootstrap serta Font Awesome** → membuat tampilan dan ikon menjadi lebih rapi tanpa perlu Node.js.

9. Buat file **resources/views/auth/register.blade.php** (buat folder auth terlebih dahulu jika belum ada), lalu isi dengan kode di bawah ini.

```
@extends('layouts.app')

@section('content')
<div class="row justify-content-center">
  <div class="col-md-6">
    <div class="card">
      <div class="card-header">Register</div> <!-- Judul form -->

      <div class="card-body">
        <form method="POST" action="{{ route('register') }}">
      @csrf <!-- Token keamanan agar form hanya bisa dikirim dari aplikasi ini -->

          <div class="mb-3">
            <label for="name" class="form-label">Name</label>
            <input id="name" type="text"
              class="form-control @error('name') is-invalid @enderror"
                name="name" value="{{ old('name') }}"
                required autocomplete="name" autofocus>
            @error('name')
              <span class="invalid-feedback" role="alert">
                <strong>{{ $message }}</strong> <!-- Pesan error dari
              validasi --> </span>
            @enderror
          </div>

          <!-- Input email -->
          <div class="mb-3">
            <label for="email" class="form-label">Email Address</label>
            <!-- Input email -->
            <input id="email" type="email"
              class="form-control @error('email') is-invalid @enderror"
                name="email" value="{{ old('email') }}"
                required autocomplete="email">
            @error('email')
              <span class="invalid-feedback" role="alert">
                <strong>{{ $message }}</strong>
              </span>
            @enderror
          </div>

          <!-- Input password -->
          <div class="mb-3">
            <label for="password" class="form-label">Password</label>
            <!-- Input password -->
            <input id="password" type="password"
              class="form-control @error('password') is-invalid @enderror"
              name="password" required autocomplete="new-password">
            <!-- Tampilkan pesan error jika ada masalah pada field 'password'
            --> @error('password')
              <span class="invalid-feedback" role="alert">
```

```
                    <strong>{{ $message }}</strong>
                  </span>
                @enderror
              </div>

              <!-- Konfirmasi password -->
              <div class="mb-3">
                <label for="password-confirm" class="form-label">Confirm
Password</label>
                <!-- Input untuk mengulang password -->
                <input id="password-confirm" type="password"
                    class="form-control"
                   name="password_confirmation" required autocomplete="new
password">
              </div>
              <div class="d-grid">
               <button type="submit" class="btn btn-primary">Register</button>
              </div>
            </form>
          </div>
        </div>
      </div>
    </div>
@endsection
```

- **@extends('layouts.app') dan @section('content')** → menggunakan layout **app.blade.php** dan mengisi bagian content.
- **action="{{ route('register') }}" dan method="POST"** → data form dikirim ke method store() pada **RegisterController**.
- **@csrf** → token keamanan agar form hanya dapat dikirim dari aplikasi ini. • **@error('name') ... @enderror** → menampilkan pesan galat di bawah input jika validasi gagal.
- **value="{{ old('name') }}"** → mengisi kembali input dengan data sebelumnya ketika validasi gagal, sehingga pengguna tidak perlu mengetik ulang.
- **name="password_confirmation"** → input konfirmasi password yang dibutuhkan oleh aturan validasi confirmed.

10. Buat file **resources/views/auth/login.blade.php**, lalu isi dengan kode di bawah ini. @extends('layouts.app')

```
@section('content')
<div class="row justify-content-center">
  <div class="col-md-6">
    <div class="card">
      <div class="card-header">Login</div>

      <div class="card-body">
        {{-- Formulir login. Data dikirim ke route login menggunakan POST --}}
        <form method="POST" action="{{ route('login') }}">
      @csrf {{-- Token keamanan agar form hanya bisa dikirim dari aplikasi ini --}}
```

```
            <div class="mb-3">
              <label for="email" class="form-label">Email Address</label>
              <input id="email" type="email"
                 class="form-control @error('email') is-invalid @enderror"
                   name="email" value="{{ old('email') }}"
                   required autocomplete="email" autofocus>
            {{-- Tampilkan pesan kesalahan jika input email bermasalah --}}
              @error('email')
                 <span class="invalid-feedback" role="alert">
              <strong>{{ $message }}</strong> {{-- Menampilkan pesan validasi --}}
                 </span>
              @enderror
            </div>

            {{-- Input untuk password --}}
            <div class="mb-3">
              <label for="password" class="form-label">Password</label>
              <input id="password" type="password"
                class="form-control @error('password') is-invalid @enderror"
                 name="password" required autocomplete="current-password">
              @error('password')
                 <span class="invalid-feedback" role="alert">
              <strong>{{ $message }}</strong> {{-- Menampilkan pesan validasi --}}
                 </span>
              @enderror
            </div>

            {{-- Checkbox untuk "Ingat Saya" --}}
            <div class="mb-3 form-check">
              <input class="form-check-input" type="checkbox"
name="remember"  id="remember"
                   {{ old('remember') ? 'checked' : '' }}>
              <label class="form-check-label" for="remember">
                 Remember Me
              </label>
            </div>

            <div class="d-grid">
             <button type="submit" class="btn btn-primary">Login</button>
            </div>
          </form>
        </div>
      </div>
    </div>
</div>
@endsection
```

- **action="{{ route('login') }}"** → data email dan password dikirim ke method store() pada **LoginController**.
- **name="remember"** → checkbox Remember Me; jika dicentang, pengguna

tetap  login walaupun browser ditutup.

11. Buat file **resources/views/dashboard.blade.php**, lalu isi dengan kode di bawah ini. @extends('layouts.app')

```
@section('content')
<div class="container">
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card">
        <div class="card-header">Dashboard</div>

        <div class="card-body">
          @if (session('status'))
            <div class="alert alert-success" role="alert">
              {{ session('status') }}
            </div>
          @endif

          You are logged in!
        </div>
      </div>
    </div>
  </div>
</div>
@endsection
```

Halaman dashboard hanya berisi pesan You are logged in! sebagai penanda bahwa pengguna berhasil login. Karena route **/dashboard** berada di dalam grup middleware auth, halaman ini tidak dapat dibuka sebelum login.

12. Buat file **resources/views/profile.blade.php**, lalu isi dengan kode di bawah ini.

```
@extends('layouts.app')


@section('content')
<div class="container">
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card">
        <div class="card-header">Profile</div>


        <div class="card-body">
          <p>Name: {{ Auth::user()->name }}</p>
          <p>Email: {{ Auth::user()->email }}</p>
          <p>Account created: {{ Auth::user()->created_at->diffForHumans()
        }}</p> </div>
      </div>
    </div>
  </div>
</div>
@endsection
```

- **Auth::user()** → mengambil data pengguna yang sedang login dari tabel users.
- **Auth::user()->name dan Auth::user()->email** → menampilkan nama dan email pengguna.
- **created_at->diffForHumans()** → menampilkan waktu pembuatan akun dalam format yang mudah dibaca, misalnya 5 minutes ago.

13. Jalankan aplikasi dengan perintah **php artisan serve**, lalu buka **http://127.0.0.1:8000** di browser dan uji fitur yang telah dibuat dengan langkah berikut:
    - **Buka http://127.0.0.1:8000/dashboard sebelum login** → Anda akan otomatis diarahkan ke halaman login oleh middleware auth.
    - **Klik menu Register, isi form, lalu klik Register** → akun tersimpan di tabel users dan Anda langsung masuk ke halaman dashboard.
    - **Klik menu Profile** → nama, email, dan waktu pembuatan akun Anda ditampilkan. • Klik Logout, lalu login kembali menggunakan email dan password yang sama. Cobalah juga memasukkan password yang salah untuk melihat pesan galat.

Jika seluruh pengujian di atas berjalan sesuai harapan, berarti alur authentication manual sudah berjalan dengan benar, yaitu form → route → controller → Auth/Hash → database → session.

### 6.5 Tugas Praktikum

1. Setelah LaporBanjir pada Modul 4 terhubung dengan database, Badan Penanggulangan Bencana Daerah (BPBD) Kabupaten Bandung menemukan masalah baru: siapa pun yang mengetahui alamat aplikasi dapat membuka halaman daftar laporan dan mengakses data warga yang melapor. BPBD meminta agar halaman pengelolaan laporan hanya dapat diakses oleh petugas yang sudah memiliki akun dan berhasil login, sedangkan warga tetap dapat mengirim laporan melalui form tanpa harus login. Oleh karena itu, kembangkan kembali project LaporBanjir dari Modul 4 dengan ketentuan sebagai berikut:
   - Authentication manual: buat fitur register, login, dan logout tanpa Breeze (seperti subbab 6.4.10) menggunakan **RegisterController** dan **LoginController**, lengkap dengan validasi input, **Hash::make()**, **Auth::attempt()**, dan **Auth::logout()**.
   - Proteksi halaman: lindungi halaman daftar laporan (dan halaman pengelolaan lainnya) menggunakan middleware auth, sedangkan halaman form pelaporan tetap dapat diakses tanpa login. Buktikan bahwa membuka halaman daftar laporan sebelum login akan diarahkan ke halaman login.
   - Navigasi: tampilkan menu Login dan Register ketika pengguna belum login, serta menu Daftar Laporan, Profil Petugas, dan Logout ketika pengguna sudah login menggunakan directive **@auth** dan **@endauth** pada layout layouts/**app.blade.php**.
   - Profil petugas: buat halaman profil yang menampilkan nama, email, dan waktu pembuatan akun petugas yang sedang login menggunakan **Auth::user()**.
   - Seeder petugas: buat seeder bernama **PetugasSeeder** yang menggunakan Faker dengan locale id_ID untuk menghasilkan minimal 5 akun petugas dummy dengan password yang di-hash, ditambah 1 akun admin BPBD dengan email dan password tetap. Jalankan seeder, lalu uji login menggunakan akun tersebut.
2. Terapkan fitur authentication pada project Tugas Akhir Pemrograman Web Semester 2 Saudara (yang telah menggunakan database pada Modul 4), dengan ketentuan sebagai berikut:
   - Pilih salah satu cara pembuatan authentication, yaitu menggunakan Laravel Breeze atau secara manual, lalu jelaskan alasan pemilihan tersebut pada laporan. • Sediakan fitur register, login, dan logout yang berjalan dengan baik, termasuk validasi input dan pesan galat yang jelas.
   - Tentukan halaman mana saja yang wajib login dan lindungi halaman tersebut menggunakan middleware auth.
   - Buat minimal 5 akun dummy menggunakan seeder dan Faker, lalu daftarkan seeder tersebut pada **DatabaseSeeder.php**.
   - Tunjukkan data akun di tabel users pada phpMyAdmin dan pastikan kolom password tersimpan dalam bentuk hash.

### 6.6 Format Pengumpulan Tugas

Laporan disusun secara berurutan dengan sistematika berikut.

**1. Halaman Sampul (Cover)**

Memuat judul praktikum, nama mahasiswa, NIM, kelas, mata kuliah, nama dosen pengampu, program studi, serta tahun akademik. Gunakan logo institusi dan tata letak yang rapi.

**2. Tujuan Praktikum**

Menuliskan tujuan yang ingin dicapai pada praktikum ini dengan kalimat sendiri, bukan menyalin ulang tujuan pembelajaran pada modul.

**3. Alat dan Bahan**

Menyebutkan spesifikasi perangkat keras yang digunakan serta perangkat lunak beserta versinya, misalnya XAMPP/Laragon, MySQL/MariaDB, phpMyAdmin, Composer, PHP, Laravel Framework, Laravel Breeze, Node.js dan NPM, Visual Studio Code, dan Git. **4. Langkah-Langkah Praktikum**

Menguraikan tahapan pengerjaan secara runtut, mulai dari instalasi Laravel Breeze (atau pembuatan authentication secara manual), pembuatan controller register dan login, pengaturan route dan middleware auth, pembuatan layout dan view (register, login, dashboard, dan profil), pembuatan seeder akun dummy dengan Faker, hingga pengujian register, login, proteksi halaman, dan logout melalui browser dan phpMyAdmin. Setiap langkah disertai potongan kode yang relevan dan penjelasan singkat.

**5. Hasil dan Dokumentasi Program**

Menampilkan tangkapan layar hasil aplikasi yang berjalan, sekurang-kurangnya halaman register, halaman login, pesan galat ketika login gagal, hasil redirect ke halaman login ketika halaman daftar laporan dibuka sebelum login, halaman daftar laporan dan profil petugas setelah login, dan tabel users berisi akun hasil seeder (dengan password dalam bentuk hash) pada LaporBanjir, serta fitur register, login, dan logout pada project Tugas Akhir. Setiap gambar diberi nomor, keterangan gambar, dan penjelasan singkat.

**6. Analisis dan Pembahasan**

Membahas perbedaan aplikasi sebelum dan sesudah menggunakan authentication, perbedaan membuat authentication dengan Laravel Breeze dan secara manual, peran session, hashing password, dan middleware auth, alasan pemilihan halaman yang wajib login, serta kendala atau galat yang ditemukan (misalnya galat 419 Page Expired, Route [login] not defined, login selalu gagal, atau npm tidak dapat dijalankan) beserta cara penyelesaiannya.

**7. Kesimpulan**

Menuliskan ringkasan capaian praktikum dan pemahaman yang diperoleh, disusun dalam beberapa kalimat yang menjawab tujuan praktikum.

**8. Lampiran berisi *Link* Repositori GitHub**

Mencantumkan tautan repositori GitHub yang dapat diakses oleh dosen pengampu dan memuat kode program lengkap, termasuk file controller authentication, route, view, dan seeder akun petugas.

**9. Presentasi Hasil Praktikum**

Mahasiswa mempresentasikan hasil praktikum di kelas, meliputi demonstrasi aplikasi yang berjalan (termasuk proses register, login, mengakses halaman yang dilindungi middleware auth, logout, serta menjalankan seeder akun petugas) dan penjelasan kode program, serta menjawab pertanyaan dari dosen pengampu.

**TERIMAKASIH**